

UNIT II – PROCESSING AND STORING STREAMING DATA

2-Mark and 10-Mark Exam Answers

Exam-oriented notes: Apache Storm, Spouts, Bolts, Topologies, Storm clusters, ZooKeeper, Nimbus, Supervisors, NoSQL storage and consistent hashing.

PART A – 2 MARK QUESTIONS

1. Analyze the advantages of distributed stream processing over centralized processing in large-scale applications.

Distributed stream processing divides the workload among multiple processing nodes. It provides higher throughput, horizontal scalability and fault tolerance, whereas centralized processing is limited by the resources and failure of a single main system.

2. Differentiate the functional responsibilities of Spouts, Bolts, and Topologies in Apache Storm.

Spout receives data and generates tuples. **Bolt** processes the tuples through filtering, transformation or aggregation. **Topology** connects Spouts and Bolts as a directed acyclic graph (DAG) defining the complete data flow.

3. Examine the influence of Storm cluster configuration parameters on performance and reliability.

Storm configuration parameters such as ZooKeeper settings, Nimbus configuration, worker settings and Netty communication settings affect task distribution, communication speed, throughput and reliability. Multiple ZooKeeper servers can improve failover.

4. Contrast local clusters and distributed clusters with respect to deployment and scalability.

A local cluster runs the topology in a single JVM and is mainly used for development and testing. A distributed cluster runs across multiple machines and supports higher processing capacity, horizontal scalability and production workloads.

5. Trace the flow of tuples through a Storm topology and identify the role of each processing stage.

Tuples flow as **Data Source** → **Spout** → **Bolt** → **Bolt** → **Output/Storage**. The Spout generates tuples, while successive Bolts process, filter, transform or aggregate them according to the topology.

6. Assess the effectiveness of consistent hashing in addressing scalability challenges in distributed storage systems.

Consistent hashing maps data keys and storage nodes into a common hash space. When nodes are added or removed, only a limited portion of data needs redistribution, thereby reducing migration and supporting scalability and load balancing.

7. Evaluate the suitability of NoSQL storage systems for handling high-velocity data streams.

NoSQL systems are suitable because they support distributed storage, horizontal scaling and flexible data models. Redis provides very fast access, Cassandra supports large distributed datasets, and MongoDB supports richly structured and geographical data.

8. Design a Storm topology for processing real-time sensor data generated by a smart grid.

A suitable topology is **Smart Grid Sensors** → **Sensor Spout** → **Processing Bolt** → **Aggregation/Analysis Bolt** → **Storage/Alert Bolt**. The Spout receives sensor streams and Bolts analyze values and generate alerts or store results.

9. Configure a Storm cluster to support fault-tolerant processing of financial transaction streams.

Use a distributed Storm cluster with **Nimbus**, multiple **Supervisors** and **ZooKeeper**. Multiple Supervisors provide processing redundancy, while ZooKeeper supports coordination and failover. Failed tasks can be restarted on available workers.

10. Determine the most suitable cluster architecture for a social-media analytics platform and justify the selection.

A distributed Storm cluster is suitable because social-media data has high volume and velocity. Multiple Supervisors and workers provide parallel processing, while Nimbus manages tasks and ZooKeeper coordinates the cluster.

11. Apply consistent hashing to optimize data distribution among storage nodes.

Map both storage nodes and data keys onto a hash ring. Each key is assigned to the appropriate node according to its position. Adding or removing a node affects only a limited range of keys, reducing redistribution.

12. Develop a storage strategy for processing and warehousing large-scale streaming data.

Use Redis for fast access to current real-time results and Cassandra for large-scale distributed storage. Replication and partitioning can provide scalability and availability for continuous streaming workloads.

PART B – 10 MARK QUESTIONS

1. Analyze the architecture of distributed streaming data processing systems and assess their advantages over centralized processing approaches.

Introduction: A distributed streaming system processes continuously arriving data using multiple interconnected processing nodes. Apache Storm is a distributed stream-processing framework in which processing is organized as topologies containing Spouts and Bolts.

Architecture: **Streaming Sources** → **Collection** → **Data Flow** → **Distributed Processing** → **Storage** → **Output/Alerts**. In Storm, Spouts provide input and Bolts perform processing. A topology defines the data-flow graph.

Data Sources: Sources include sensors, websites, applications, social media and financial systems that continuously generate events.

Data Ingestion: The ingestion layer collects incoming events and supplies them to the streaming infrastructure.

Distributed Processing: The workload is divided among multiple workers. Different parts of the stream can therefore be processed in parallel.

Cluster Coordination: A distributed Storm deployment uses Nimbus, Supervisors and ZooKeeper. Nimbus manages task distribution, Supervisors execute tasks and ZooKeeper provides coordination.

Scalability: Additional processing nodes can be added when data rates increase, providing horizontal scalability.

Throughput: Parallel processing across workers enables the system to handle larger event rates than a single centralized processor.

Fault Tolerance: Redundant nodes and task recovery allow processing to continue when individual components fail.

Conclusion: Distributed stream processing is better suited to large-scale real-time applications because it provides scalability, parallelism, higher throughput and fault tolerance.

2. Differentiate the functional roles of Spouts, Bolts, and Topologies in Apache Storm and examine their interactions during stream processing.

Introduction: Storm represents streaming applications using a topology that connects Spouts and Bolts.

Spout: A Spout is an input component. It receives data from external sources and emits tuples into the topology.

Bolt: A Bolt processes tuples received from Spouts or other Bolts. It can perform filtering, transformation, aggregation, calculation or storage operations.

Topology: A Topology defines the complete processing structure by connecting Spouts and Bolts as a directed acyclic graph.

Interaction: **External Source** → **Spout** → **Bolt 1** → **Bolt 2** → **Output**. The Spout produces tuples, which are passed to downstream Bolts for processing.

Example: For temperature monitoring, the Spout reads sensor values, Bolt 1 validates them, Bolt 2 checks thresholds, and Bolt 3 generates alerts.

Benefits: This structure provides modular processing, parallel execution and continuous stream processing.

Conclusion: Spout = input, Bolt = processing, and Topology = complete data-flow structure.

3. Evaluate the impact of cluster configuration parameters on the performance, scalability, and fault tolerance of Apache Storm deployments.

Introduction: Storm configuration controls communication, task distribution and cluster behavior, directly affecting performance and reliability.

ZooKeeper: ZooKeeper stores coordination information and helps Storm components coordinate. Multiple ZooKeeper servers can improve availability.

Nimbus: Nimbus distributes and manages tasks across Supervisors and supports rebalancing and recovery.

Supervisors: Supervisors execute assigned tasks. Increasing the number of Supervisors increases available processing resources.

Workers: Worker processes execute topology components. Appropriate worker allocation improves parallelism, throughput and CPU utilization.

Netty: Netty can provide communication between worker processes. Worker-thread and buffer settings affect communication performance.

Retries and Timeouts: Retry and timeout parameters determine how the system responds to communication or processing failures.

Scalability: Adding Supervisors and workers allows the system to handle increasing data rates.

Fault Tolerance: Replication, multiple coordination servers and task recovery reduce the effect of component failures.

Conclusion: Correct cluster configuration is essential for high throughput, scalability, low latency and reliable processing.

4. Contrast local clusters and distributed clusters with respect to deployment complexity, resource utilization, and operational scalability.

Introduction: Storm supports local execution for development/testing and distributed execution for production workloads.

Local Cluster: A local cluster runs the complete topology in a single JVM. It is simple to configure and debug.

Local Resource Use: It uses the resources of one machine, so processing capacity and scalability are limited.

Distributed Cluster: A distributed cluster executes components across multiple machines and uses cluster services such as Nimbus, Supervisors and ZooKeeper.

Deployment Complexity: Distributed deployment is more complex because multiple machines and coordination services must be configured.

Resource Utilization: Distributed clusters use CPU, memory and network resources across several machines.

Scalability: New Supervisors and workers can be added to increase capacity.

Fault Tolerance: Distributed clusters can recover from failures using redundancy and task reassignment.

Comparison: Local: simple, single JVM, testing, limited scalability. Distributed: complex, multiple machines, production, high scalability.

Conclusion: Use local mode for development and distributed mode for large-scale production stream processing.

5. Assess the effectiveness of consistent hashing in supporting scalability and load balancing within distributed storage systems.

Introduction: Consistent hashing is a technique for distributing data across nodes while minimizing data movement when nodes change.

Hash Ring: Storage nodes and data keys are mapped into a common hash space, often visualized as a ring.

Data Assignment: A data key is assigned to the appropriate node according to its hash position.

Node Addition: When a new node is added, only a limited range of keys needs to be reassigned.

Node Removal: When a node is removed, its affected keys can be reassigned to other nodes according to the hashing scheme.

Scalability: Limited redistribution makes it easier to add or remove storage capacity.

Load Balancing: Keys are spread among nodes, helping distribute storage and request load and reducing hotspots.

Performance: Reduced data migration lowers network traffic and avoids large-scale redistribution overhead.

Fault Tolerance: Consistent hashing helps data placement, but replication is still needed to protect against node failures.

Conclusion: Consistent hashing is effective for dynamic distributed storage because it improves scalability, load distribution and migration efficiency.

6. Design a Storm topology capable of processing real-time environmental sensor data and generating alerts for abnormal conditions.

Problem: Environmental sensors continuously produce temperature, humidity, pressure and air-quality data. The system must detect abnormal readings quickly.

Architecture: **Environmental Sensors** → **Sensor Spout** → **Validation Bolt** → **Analysis Bolt** → **Alert Bolt** → **Alert System**. Normal readings can also be sent to storage.

Sensor Spout: Receives environmental readings and emits tuples such as sensor ID, timestamp and measurement.

Validation Bolt: Checks missing, invalid or inconsistent readings before further processing.

Analysis Bolt: Compares measurements with thresholds or normal patterns and identifies abnormal conditions.

Alert Bolt: Generates an alert when an abnormal condition is detected.

Storage: Current values can be kept in fast storage such as Redis, while larger distributed datasets can be stored in Cassandra.

Example: If temperature exceeds a defined threshold, the Analysis Bolt marks it abnormal and sends the event to the Alert Bolt.

Advantages: Continuous monitoring, low-latency detection, scalable processing and support for distributed deployment.

Conclusion: The topology converts raw sensor streams into validated readings, analyzed conditions and immediate alerts.

7. Stock exchange: Develop a Storm-based distributed stream processing solution and recommend an appropriate storage strategy using consistent hashing.

Requirements: The stock exchange requires millions of events per second, low latency, continuous availability and fault tolerance.

Architecture: **Trading Sources** → **Collection/Kafka** → **Storm Spouts** → **Validation Bolts** → **Aggregation/Analysis Bolts** → **Fast Output + Distributed Storage**.

Spouts: Receive trading events containing stock symbol, price, quantity and timestamp.

Processing Bolts: Validation Bolts check events; aggregation Bolts calculate volumes and prices; analysis Bolts detect unusual activity.

Distributed Cluster: Use Nimbus, multiple Supervisors, workers and ZooKeeper. Parallel workers increase throughput.

Low Latency: Use parallel processing, efficient data flow and fast access to current results.

Fault Tolerance: Use multiple Supervisors, coordination redundancy, task recovery and replicated storage.

Consistent Hashing: Hash transaction keys such as stock symbol and transaction ID and distribute them among storage nodes.

Storage: Cassandra is suitable for large-scale distributed storage, while Redis can provide fast access to current results.

Benefits: Consistent hashing reduces data movement when storage nodes are added or removed and distributes load across nodes.

Conclusion: Storm provides continuous distributed processing, while consistent-hashing-based distributed storage provides scalable and efficient storage.

8. Global video-streaming service: Apply consistent hashing principles to construct a scalable storage architecture and evaluate its impact on load distribution and system performance.

Problem: A global video service stores activity logs across hundreds of servers. Frequent server changes cause excessive data migration.

Hash Ring: Create a logical hash ring and map storage servers onto the ring.

User Keys: Hash user activity keys such as user ID, session ID or activity ID and map them to the ring.

Data Placement: Each key is assigned to the appropriate storage node according to the ring order.

Server Addition: When a new server is added, only the affected range of keys is migrated rather than the complete dataset.

Server Removal: When a server is removed, its affected keys are reassigned to other nodes.

Load Distribution: Keys are distributed across many nodes, improving utilization and reducing hotspots.

Performance: Reduced migration lowers network traffic, storage overhead and temporary performance degradation.

Fault Tolerance: Replication should be combined with consistent hashing so that node failures do not cause data loss or service interruption.

Conclusion: Consistent hashing provides a scalable storage architecture with lower data movement, better load distribution and more stable performance during server changes.

UNIT II – QUICK REVISION

Spout: Input component; emits tuples.

Bolt: Processes tuples.

Topology: DAG connecting Spouts and Bolts.

Nimbus: Manages/distributes Storm tasks.

Supervisor: Executes tasks/workers.

ZooKeeper: Cluster coordination and failover support.

Local Cluster: Single JVM; development/testing.

Distributed Cluster: Multiple machines; production/scalability.

Redis: Very fast access to current data.

Cassandra: Distributed large-scale storage.

MongoDB: Rich/geographical data.

Consistent Hashing: Hash keys + nodes → distributed storage with reduced data migration.